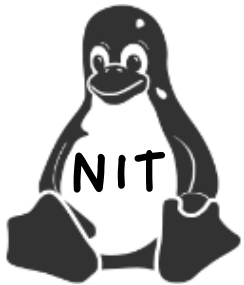


Welcome
Everyone



MASTERING
LINUX!
Day-65
Linux Processes



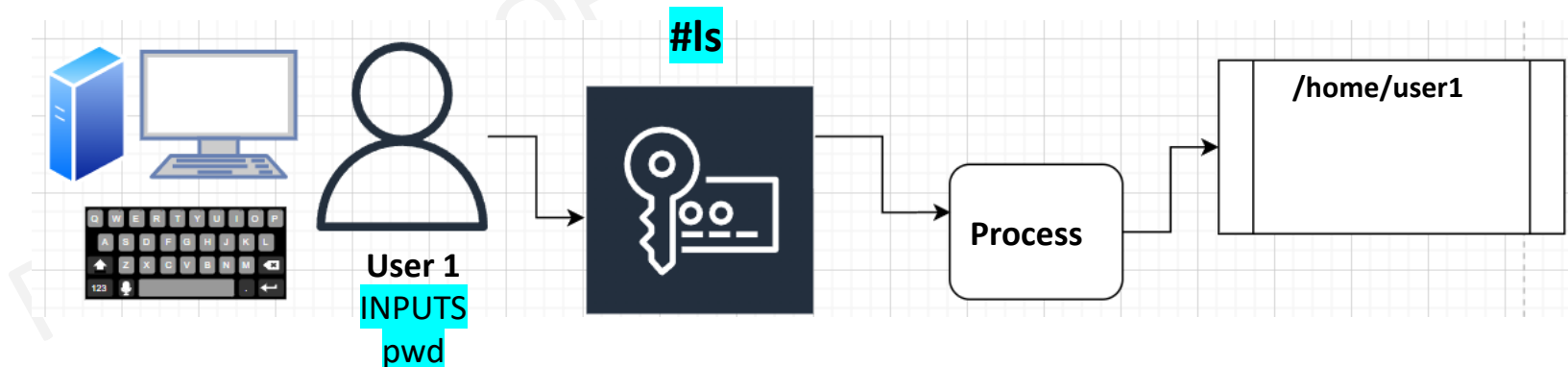


ACCESS CONTROL LIST

PROCESSES IN LINUX

Process : Set of instructions loaded in a memory. When a user “executes” a command in linux. A new process is initiated by the kernel allocating certain amount of memory and cpu utilization to the process. OS system tracks a process via a process ID (PID). PID is always unique for each process

- ❖ ps report a snapshot of the current processes
- ❖ #ps -ef
- ❖ #ps aux
- ❖ ps -a All terminals
- ❖ ps -e list of all the processes
- ❖ ps -o (customer properties; customized properties)
- ❖ ps -ao tty,comm,pid,%mem,%cpu



PS -EF

```
[nitadmin1@System-Admin ~]$ ps -ef
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	Jan23	?	00:00:15	/usr/lib/systemd/systemd --s
root	2	0	0	Jan23	?	00:00:00	[kthreadd]
root	3	2	0	Jan23	?	00:00:00	[pool_workqueue_]
root	4	2	0	Jan23	?	00:00:00	[kworker/R-rcu_g]
root	5	2	0	Jan23	?	00:00:00	[kworker/R-rcu_p]
root	6	2	0	Jan23	?	00:00:00	[kworker/R-slub_]
root	7	2	0	Jan23	?	00:00:00	[kworker/R-netns]

UID – User ID

PID - Process ID

PPID – Parent Process ID

C - CPU Utilization

STIME – Time when Process Started

TTY - Terminal Type Source of input is **Kernel (=?)**

CMD – Command

```
[nitadmin1@System-Admin ~]$ ps
```

PID	TTY	TIME	CMD
7081	tty2	00:00:00	bash
7106	tty2	00:00:00	ps

```
[nitadmin1@System-Admin ~]$ ps -ef | grep -i tty
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1274	727	0	Jan23	tty1	00:00:00	-bash
nitadmi+	7081	7077	0	07:11	tty2	00:00:00	-bash
nitadmi+	7119	7081	0	07:13	tty2	00:00:00	ps -ef
nitadmi+	7120	7081	0	07:13	tty2	00:00:00	grep --color=auto -i tty

PS AND NICE

```
[nitadmin1@System-Admin ~]$ ps -ao tty,comm,pid,nice,%mem,%cpu
```

TT	COMMAND	PID	NI	%MEM	%CPU
pts/0	ps	8224	0	0.4	0.0

```
[nitadmin1@System-Admin ~]$ ps -ao tty,comm,pid,nice,%mem,%cpu
```

TT	COMMAND	PID	NI	%MEM	%CPU
pts/0	ps	8224	0	0.4	0.0

```
[nitadmin1@System-Admin ~]$ renice -4 8224
```

```
renice: failed to get priority for 8224 (process ID): No such process
```

•Nice Value Range:

- **-20**: Highest priority (most favorable scheduling)
- **0**: Default priority
- **+19**: Lowest priority (least favorable scheduling)

How the nice value affects priority:

- The **lower the nice value**, the higher the priority of the process.
- The **higher the nice value**, the lower the priority of the process.

#top

#nice -n -10 command

PS AUX

- A common display listing (options **aux**) displays **all processes**, with detailed information.
- “grep” is commonly used in format where we are searching for a specific process

```
Last login: Fri Jan 24 07:11:50 2025
[nitadmin1@System-Admin ~]$ ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	2.0	173384	16280	?	Ss	Jan23	0:15	/usr/lib/syst
root	2	0.0	0.0	0	0	?	S	Jan23	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	S	Jan23	0:00	[pool_workque
root	4	0.0	0.0	0	0	?	I<	Jan23	0:00	[kworker/R-rc
root	5	0.0	0.0	0	0	?	I<	Jan23	0:00	[kworker/R-rc
root	6	0.0	0.0	0	0	?	I<	Jan23	0:00	[kworker/R-sl
root	7	0.0	0.0	0	0	?	I<	Jan23	0:00	[kworker/R-ne
root	11	0.0	0.0	0	0	?	I<	Jan23	0:00	[kworker/R-rc

```
[nitadmin1@System-Admin ~]$ ps aux | grep sleep
```

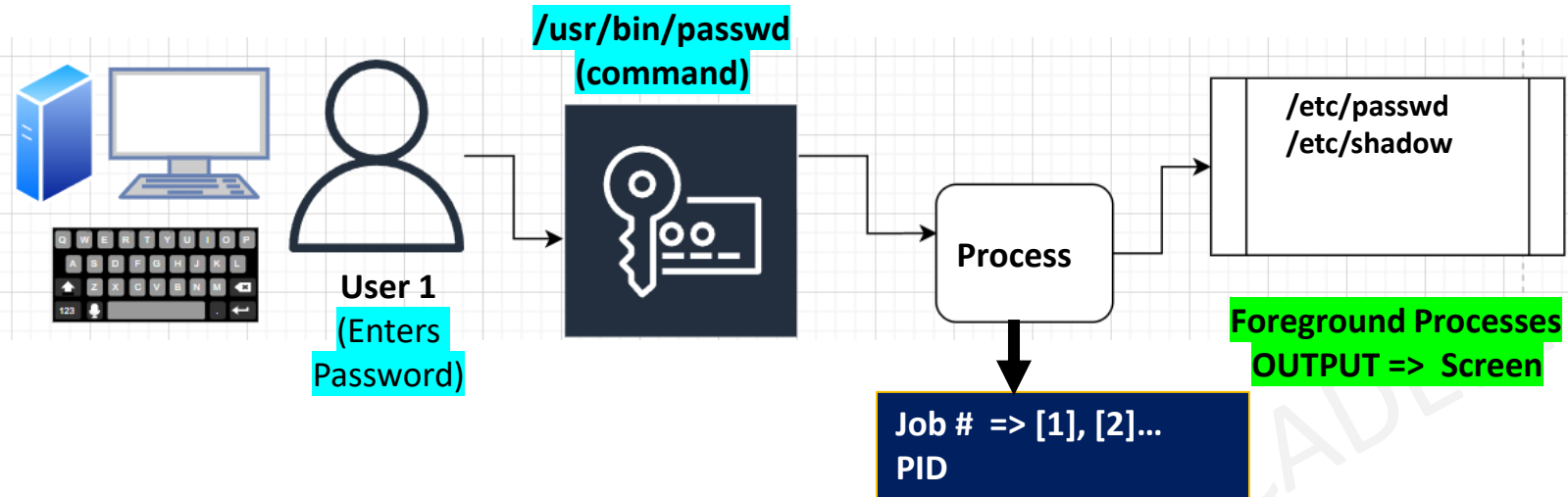
```
nitadmi+ 8147 0.0 0.2 5584 1920 pts/0 S 10:32 0:00 sleep 120
```

```
nitadmi+ 8149 0.0 0.2 6408 2304 pts/0 S+ 10:32 0:00 grep --color=auto sleep
```

PROCESS STATS

The **STAT (Statistics)** given in **TOP** and **#ps aux** command are defined as below:
Here's what that means:

1. **S** (interruptible sleep): The process is waiting for some event to complete (e.g., waiting for input/output). It is not actively running but can be woken up if an event it is waiting on occurs.
2. **R** (running): The process is actively running or ready to run.
3. **Z** (zombie): The process has finished execution but still has an entry in the process table because its parent hasn't yet read its exit status.
4. **T** (stopped): The process has been stopped (usually by a signal).



Foreground Process

When User 1 gives an input using a keyboard a process starts to run. Every process that you start runs in the foreground.

For example, `ls -l` command is fired no other commands can be run. Why? Because you won't get the line prompt to type anything

```
[nitadmin1@System-Admin ~]$ ls
khuban.txt  linux.txt  package.txt  testdir
```

Background Process

This process runs in the background allowing the user to input more commands while a process is running.

This process can be started by adding an **ampersand (&)** at the end of the command

```
[nitadmin1@System-Admin ~]$ ls &
[1] 7906
[nitadmin1@System-Admin ~]$ khuban.txt  linux.txt  package.txt  testdir
```


PRACTICE LAB

STEP 1: Start a Foreground process

```
[nitadmin1@System-Admin ~]$ sleep 10
```

STEP 2: Press Cntrl + z to suspend the process and move to Background

```
^Z
```

```
[1]+  Stopped                sleep 10
```

STEP 3: Run bg to resume the process stopped in background

```
nitadmin1@System-Admin ~]$ bg
```

```
[1]+ sleep 10 &
```

Step 4: Press fg to terminate the process

```
[nitadmin1@System-Admin ~]$ fg
```

```
-bash: fg: job has terminated
```

```
[1]+  Done                  sleep 10
```

NOTE: CNTRL + C terminates a Foreground Process

```
[nitadmin1@System-Admin ~]$ jobs
[nitadmin1@System-Admin ~]$ sleep 10
^Z
[1]+  Stopped                sleep 10
[nitadmin1@System-Admin ~]$ jobs
[1]+  Stopped                sleep 10
[nitadmin1@System-Admin ~]$ fg %1
sleep 10
```

PRACTICE LAB

What if your process is running and you need to log out but not terminate your process

That is a “no hang up” = **nohup** command

```
[nitadmin1@System-Admin ~]$ nohup ./helloscript.sh &
```

NOTE: This will save an output to => **nohup.out**

NOTE: To totally remove a program from the Shell's Process Table use “**disown**” command

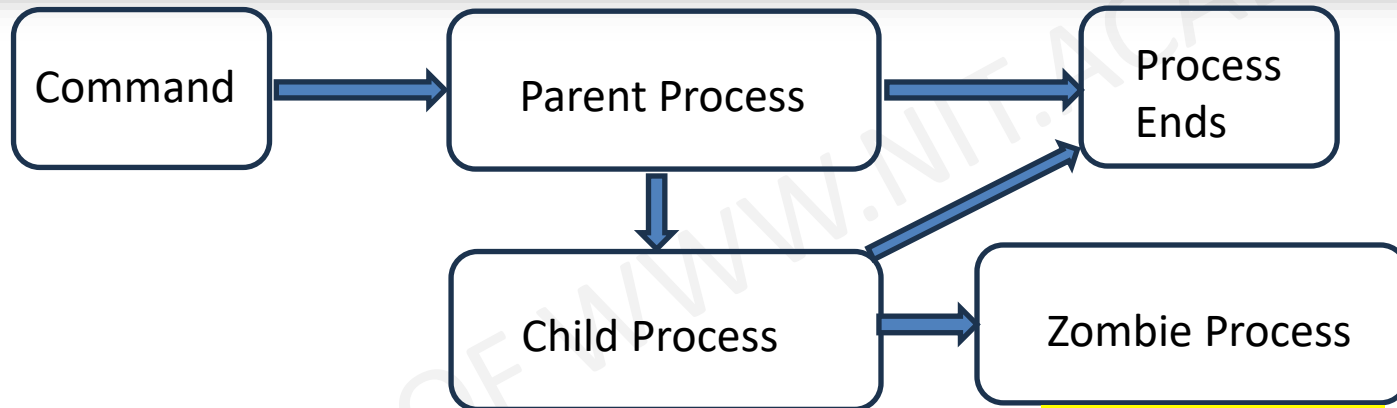
```
[nitadmin1@System-Admin ~]$ sleep 10
^Z
[1]+  Stopped                  sleep 10
[nitadmin1@System-Admin ~]$ bg
[1]+  sleep 10 &
[nitadmin1@System-Admin ~]$ disown
[nitadmin1@System-Admin ~]$ jobs
```

- Process stat codes: STAT or 'S' display to describe the state of a process
- D- uninterruptible sleep (usually IO)
 - R – This is a state where a process is either in running or ready to run.
 - S- Interruptible Sleep (Waiting for an event to complete)
 - T- Stopped, Once the process is completed, this state occurs. This process can be restarted
 - Z- Defunct (Zombie): In this state, the process will be terminated and the information will still be available in the process table.
 - W- Paging

```
[nitadmin1@System-Admin ~]$ sleep 100
^Z
[1]+  Stopped          sleep 100
[nitadmin1@System-Admin ~]$ bg
[1]+  sleep 100 &
[nitadmin1@System-Admin ~]$ ps aux | grep sleep
nitadmi+  8164  0.0  0.2  5584 1920 pts/0   S   10:35   0:00 sleep 100
nitadmi+  8176  1.0  0.2  6408 2304 pts/0   S+  10:36   0:00 grep --
color=auto sleep
[nitadmin1@System-Admin ~]$ cat /proc/8164/status
Name: sleep
Umask: 0022
State: S (sleeping)
```

ZOMBIE PROCESS

```
[nitadmin1@System-Admin ~]$ ps -ef | grep sshd
root          716          1  0 Jan23 ?           00:00:00 sshd: /usr/sbin/sshd -D [lis
tener] 0 of 10-100 startups
root          7782        716  0 09:26 ?           00:00:00 sshd: nitadmin1 [priv]
nitadmini+    7786        7782  0 09:26 ?           00:00:00 sshd: nitadmin1@pts/0
nitadmini+    8335        7787  0 11:07 pts/0       00:00:00 grep --color=auto sshd
```



How to “Kill” a Process?

In Linux, the kill command is used to send signals to processes, and signals control the behavior of processes. The most common signals are

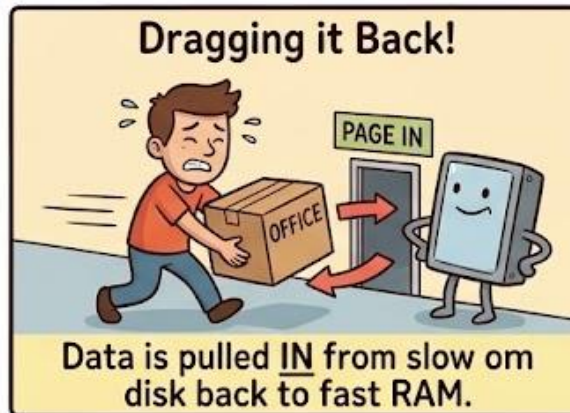
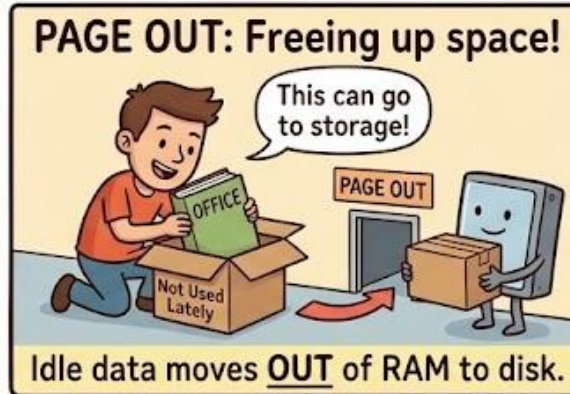
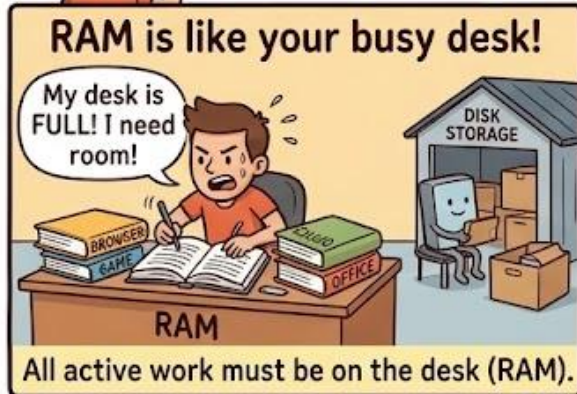
SIGTERM (15) and SIGKILL (9). They are used to stop (kill) processes, but they differ significantly in how they work.

- **#kill -15 pid => gracefully**
- **#kill -9 pid => forcefully**

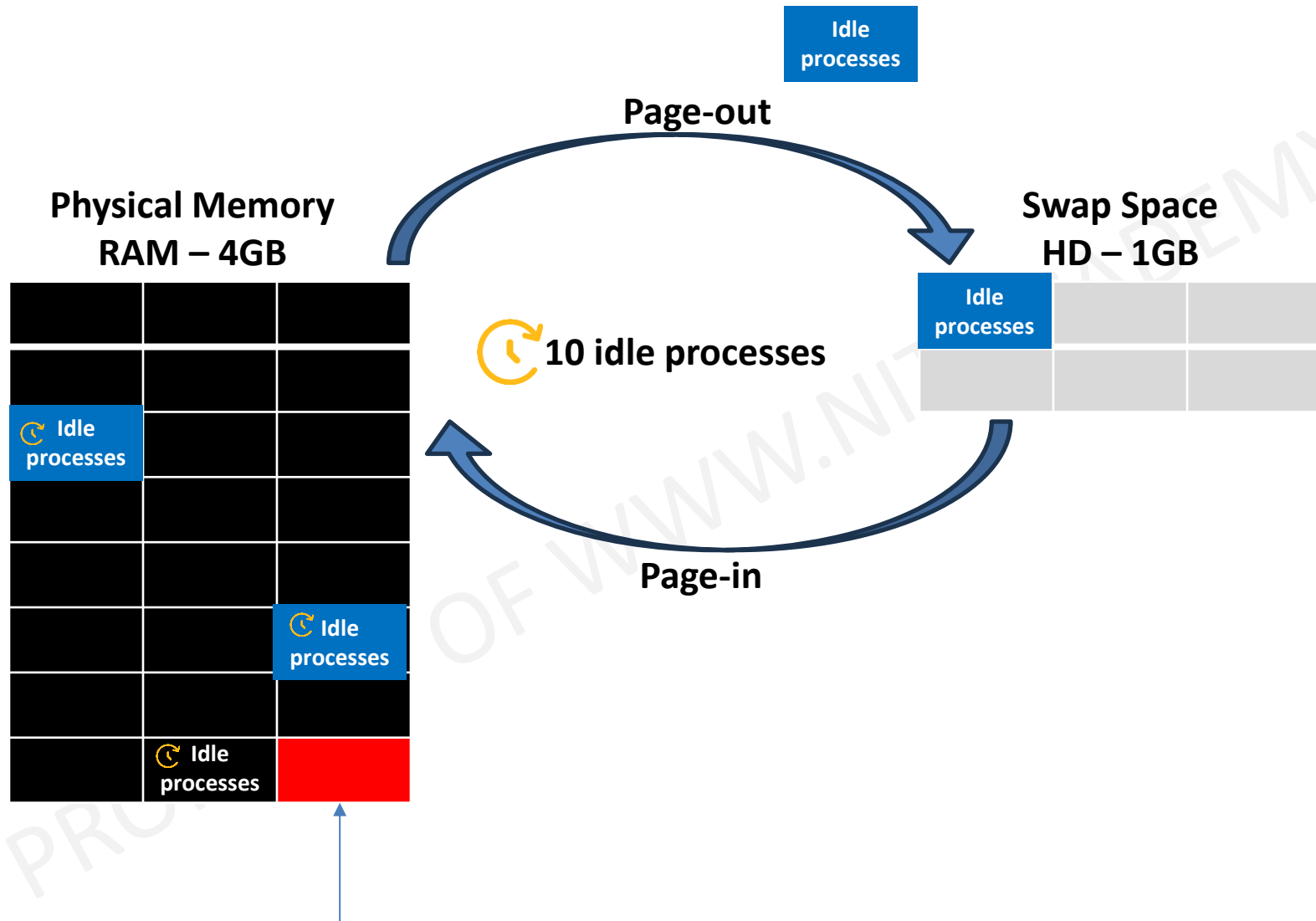
#ps aux | grep 'Z'



UNDERSTANDING MEMORY SWAP!



SWAP/SWAPINESS



SWAP SPACE

- **Swap space** is a portion of the hard disk used as **virtual memory** when the system's **RAM** is full. If RAM runs out, Linux moves **inactive pages** from RAM to swap to **prevent crashes** and keep processes running.
- **Prevents System Crashes:** If RAM is exhausted, swap helps avoid "Out of Memory" (OOM) errors.
Improves Performance: Even if RAM is full, swap allows the system to keep running (though slower).
Allows Hibernation: The system dumps RAM contents into swap during hibernation
- **Swappiness** -> `sysctl vm.swappiness=60` (Modify the swappiness value that controls how often Linux uses swap – 10 means use **swap only when necessary**, 60 is default)